MINI-PROJECT

Program 1 — Secret Code Encoder

python

```
# Secret Code Encoder
# Shifts each letter by 3 positions (Caesar Cipher)

def encode(text, shift=3):
    result = ""
    for char in text:
        if char.isalpha():
            # Keep uppercase/lowercase, wrap around with % 26
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base + shift) % 26 + base)
        else:
            result += char  # keep spaces and symbols as-is
    return result

def decode(text, shift=3):
    return encode(text, -shift)  # decoding = reverse shift

message = "Hello World"
encoded = encode(message)
decoded = decode(encoded)

print(f"Original : {message}")
print(f"Encoded  : {encoded}")
print(f"Decoded  : {decoded}")
```

Program 2 — Personal Diary with Timestamps

python

```
# Personal Diary — appends entries with date and time

import os
from datetime import datetime

DIARY_FILE = "diary.txt"

def write_entry(entry):
    timestamp = datetime.now().strftime("%d %B %Y, %I:%M %p")
    with open(DIARY_FILE, "a") as f:
        f.write(f"\n[{timestamp}]\n{entry}\n{'-'*40}\n")
    print("Entry saved!")

def read_diary():
```

```
    if not os.path.exists(DIARY_FILE):
        print("No diary entries yet.")
        return
    with open(DIARY_FILE, "r") as f:
        print(f.read())

# Write two entries then read the whole diary
write_entry("Today I learned about Python file handling. It was fun!")
write_entry("I built my first diary app. Feeling proud.")
read_diary()
```

Program 3 — Word Frequency Counter

python

```
# Reads a text and tells you how many times each word appears

from collections import Counter

def count_words(text):
    # Lowercase and split into words, strip punctuation
    words = [word.strip(".,!?;:\"'").lower() for word in text.split()]
    return Counter(words)

def show_top_words(counter, n=5):
    print(f"\nTop {n} most used words:")
    for word, count in counter.most_common(n):
        bar = "#" * count  # visual bar
        print(f"  {word:<15} {bar} ({count})")

text = """
Python is great. Python is easy to learn.
Python is used for web, data, and AI.
Learning Python is the best decision.
"""

freq = count_words(text)
show_top_words(freq)
```

Program 4 — Student Grade Calculator

python

```
# Calculates grade, percentage, and saves result to a file

def calculate_grade(marks):
    avg = sum(marks) / len(marks)
    if avg >= 90:   grade = "A+"
    elif avg >= 80: grade = "A"
    elif avg >= 70: grade = "B"
    elif avg >= 60: grade = "C"
    else:           grade = "F"
    return round(avg, 2), grade
```

```python
def save_result(name, avg, grade, filename="results.txt"):
    with open(filename, "a") as f:
        f.write(f"{name} | Average: {avg}% | Grade: {grade}\n")
    print(f"Result saved for {name}.")

def show_results(filename="results.txt"):
    with open(filename, "r") as f:
        print("\n--- All Results ---")
        print(f.read())

# Test with sample students
students = {
    "Alice": [92, 88, 95, 91],
    "Bob":   [70, 65, 72, 68],
    "Carol": [55, 60, 50, 58],
}

for name, marks in students.items():
    avg, grade = calculate_grade(marks)
    save_result(name, avg, grade)

show_results()
```

Program 5 — Number to Words Converter

python

```python
# Converts a number like 247 into "Two Hundred Forty Seven"

ones  = ["", "One", "Two", "Three", "Four", "Five",
         "Six", "Seven", "Eight", "Nine", "Ten",
         "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen",
         "Sixteen", "Seventeen", "Eighteen", "Nineteen"]

tens  = ["", "", "Twenty", "Thirty", "Forty", "Fifty",
         "Sixty", "Seventy", "Eighty", "Ninety"]

def number_to_words(n):
    if n == 0:
        return "Zero"
    if n < 20:
        return ones[n]
    if n < 100:
        rest = " " + ones[n % 10] if n % 10 != 0 else ""
        return tens[n // 10] + rest
    if n < 1000:
        rest = " " + number_to_words(n % 100) if n % 100 != 0 else ""
        return ones[n // 100] + " Hundred" + rest
    return "Number too large"

# Test
for num in [0, 5, 13, 42, 100, 256, 999]:
    print(f"{num:>4} → {number_to_words(num)}")
```

Program 6 — Expense Tracker

python

```
# Add expenses, save to JSON, show total and category summary

import json
import os

FILE = "expenses.json"

def load_expenses():
    if os.path.exists(FILE):
        with open(FILE, "r") as f:
            return json.load(f)
    return []

def add_expense(category, amount, note=""):
    expenses = load_expenses()
    expenses.append({
        "category": category,
        "amount": amount,
        "note": note
    })
    with open(FILE, "w") as f:
        json.dump(expenses, f, indent=2)
    print(f"Added: {category} — ₹{amount}")

def show_summary():
    expenses = load_expenses()
    totals = {}
    for e in expenses:
        totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]

    print("\n--- Expense Summary ---")
    for cat, total in totals.items():
        print(f"  {cat:<15} ₹{total}")
    print(f"  {'TOTAL':<15} ₹{sum(totals.values())}")

# Sample entries
add_expense("Food",      250, "Lunch")
add_expense("Transport", 80,  "Bus")
add_expense("Food",      150, "Dinner")
add_expense("Books",     500, "Python book")
show_summary()
```

Program 7 — Fibonacci Pattern Generator

python

```
# Generates Fibonacci sequence and saves it to a file

def fibonacci(n):
    sequence = [0, 1]
```

```
    for _ in range(n - 2):
        sequence.append(sequence[-1] + sequence[-2])
    return sequence[:n]

def save_to_file(sequence, filename="fibonacci.txt"):
    with open(filename, "w") as f:
        f.write("Fibonacci Sequence\n")
        f.write("=" * 30 + "\n")
        for i, num in enumerate(sequence, 1):
            f.write(f"  Term {i:>3} : {num}\n")
    print(f"Saved {len(sequence)} terms to {filename}")

def find_even_terms(sequence):
    # Filter only even Fibonacci numbers
    return [n for n in sequence if n % 2 == 0]

seq = fibonacci(20)
print("Sequence:", seq)
print("Even terms:", find_even_terms(seq))
save_to_file(seq)
```

Program 8 — Contact Book

python

```
# Save, search, and delete contacts stored in a JSON file

import json
import os

FILE = "contacts.json"

def load():
    return json.load(open(FILE)) if os.path.exists(FILE) else {}

def save(contacts):
    with open(FILE, "w") as f:
        json.dump(contacts, f, indent=2)

def add_contact(name, phone, email):
    contacts = load()
    contacts[name] = {"phone": phone, "email": email}
    save(contacts)
    print(f"Contact '{name}' saved.")

def search_contact(name):
    contacts = load()
    if name in contacts:
        info = contacts[name]
        print(f"\nName  : {name}")
        print(f"Phone : {info['phone']}")
        print(f"Email : {info['email']}")
    else:
        print("Contact not found.")
```

```
def delete_contact(name):
    contacts = load()
    if name in contacts:
        del contacts[name]
        save(contacts)
        print(f"'{name}' deleted.")
    else:
        print("Contact not found.")

# Demo
add_contact("Alice", "9876543210", "alice@email.com")
add_contact("Bob",   "9123456780", "bob@email.com")
search_contact("Alice")
delete_contact("Bob")
```

Program 9 — Password Strength Checker

python

```
# Checks how strong a password is using multiple rule functions

import re

def has_uppercase(pwd):    return bool(re.search(r'[A-Z]', pwd))
def has_lowercase(pwd):    return bool(re.search(r'[a-z]', pwd))
def has_digit(pwd):        return bool(re.search(r'\d', pwd))
def has_special(pwd):      return bool(re.search(r'[!@#$%^&*]', pwd))
def is_long_enough(pwd):   return len(pwd) >= 8

def check_strength(password):
    rules = {
        "Uppercase letter" : has_uppercase(password),
        "Lowercase letter" : has_lowercase(password),
        "Contains digit"   : has_digit(password),
        "Special character": has_special(password),
        "8+ characters"    : is_long_enough(password),
    }

    passed = sum(rules.values())

    print(f"\nPassword: {password}")
    for rule, ok in rules.items():
        status = "✓" if ok else "✗"
        print(f"  [{status}] {rule}")

    if   passed == 5: print("Strength: STRONG")
    elif passed >= 3: print("Strength: MEDIUM")
    else:             print("Strength: WEAK")

# Test
check_strength("hello")
check_strength("Hello123")
check_strength("Hello@123")
```

Program 10 — Quiz Game with Score Saved to File

python

```
# Multiple choice quiz — score is saved to a leaderboard file

import json
import os

SCORE_FILE = "leaderboard.json"

questions = [
    {"q": "What does CPU stand for?",
     "options": ["A. Central Process Unit", "B. Central Processing Unit",
                 "C. Computer Personal Unit"],
     "answer": "B"},
    {"q": "Which language is this course in?",
     "options": ["A. Java", "B. C++", "C. Python"],
     "answer": "C"},
    {"q": "What keyword defines a function in Python?",
     "options": ["A. func", "B. def", "C. define"],
     "answer": "B"},
]

def run_quiz(player_name):
    score = 0
    for i, q in enumerate(questions, 1):
        print(f"\nQ{i}: {q['q']}")
        for opt in q["options"]:
            print(f"  {opt}")
        ans = input("Your answer (A/B/C): ").strip().upper()
        if ans == q["answer"]:
            print("Correct!")
            score += 1
        else:
            print(f"Wrong! Answer was {q['answer']}")
    return score

def save_score(name, score):
    data = json.load(open(SCORE_FILE)) if os.path.exists(SCORE_FILE) else []
    data.append({"name": name, "score": score, "total": len(questions)})
    with open(SCORE_FILE, "w") as f:
        json.dump(data, f, indent=2)
    print(f"\n{name} scored {score}/{len(questions)}. Saved!")

name = input("Enter your name: ")
score = run_quiz(name)
save_score(name, score)
```

Program 11 — File Backup System

python

```python
# Copies all .txt files from a folder into a backup/ subfolder

import os
import shutil
from datetime import datetime

def create_backup(source_folder=".", backup_folder="backup"):
    # Create backup folder if it doesn't exist
    os.makedirs(backup_folder, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    backed_up = 0

    for filename in os.listdir(source_folder):
        if filename.endswith(".txt"):
            src  = os.path.join(source_folder, filename)
            # Add timestamp to backup filename so old backups aren't overwritten
            dest = os.path.join(backup_folder, f"{timestamp}_{filename}")
            shutil.copy2(src, dest)
            print(f"Backed up: {filename}")
            backed_up += 1

    if backed_up == 0:
        print("No .txt files found to back up.")
    else:
        print(f"\nTotal files backed up: {backed_up}")

create_backup()
```

Program 12 — Random Story Generator

python

```python
# Builds a random story by picking words from lists using random module

import random

def generate_story():
    heroes    = ["a young coder", "an old wizard", "a curious student"]
    actions   = ["discovered", "built", "accidentally deleted", "debugged"]
    objects   = ["a time machine", "an AI robot", "a secret database", "a magic function"]
    outcomes  = ["and changed the world.", "but nobody noticed.", "and got extra marks.",
                 "and won a hackathon."]

    # Pick one from each category randomly
    hero    = random.choice(heroes)
    action  = random.choice(actions)
    obj     = random.choice(objects)
    outcome = random.choice(outcomes)

    return f"Once upon a time, {hero} {action} {obj} {outcome}"

def save_stories(n, filename="stories.txt"):
    with open(filename, "w") as f:
        for i in range(1, n + 1):
```

```
            story = generate_story()
            f.write(f"Story {i}: {story}\n")
            print(f"Story {i}: {story}")
    print(f"\nAll {n} stories saved to {filename}")

save_stories(5)
```

Program 13 — Prime Number Finder with Module

python

```
# math_tools.py — a reusable module for number utilities

import math

def is_prime(n):
    if n < 2:
        return False
    # Only check divisors up to square root (efficient)
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def primes_in_range(start, end):
    return [n for n in range(start, end + 1) if is_prime(n)]

def save_primes(primes, filename="primes.txt"):
    with open(filename, "w") as f:
        f.write(f"Prime numbers: {primes}\n")
        f.write(f"Total found  : {len(primes)}\n")

# Main program
primes = primes_in_range(1, 100)
print(f"Primes from 1 to 100:\n{primes}")
print(f"Count: {len(primes)}")
save_primes(primes)
```

Program 14 — Typing Speed Tester

python

```
# Measures how fast you type a sentence and saves your best score

import time
import json
import os

SCORES_FILE = "typing_scores.json"

SENTENCES = [
    "Python makes programming fun and easy.",
    "Functions help you reuse code efficiently.",
```

```
    "File handling lets you store data permanently.",
]

def load_best():
    if os.path.exists(SCORES_FILE):
        with open(SCORES_FILE, "r") as f:
            return json.load(f).get("best_wpm", 0)
    return 0

def save_best(wpm):
    with open(SCORES_FILE, "w") as f:
        json.dump({"best_wpm": wpm}, f)

def run_test(sentence):
    print("\nType this sentence:")
    print(f"  >> {sentence}\n")
    input("Press ENTER when ready...")

    start = time.time()
    typed = input("Start typing: ")
    elapsed = time.time() - start

    words    = len(sentence.split())
    wpm      = round((words / elapsed) * 60, 2)
    accuracy = round(sum(a == b for a, b in zip(typed, sentence)) / len(sentence) * 100,
1)

    print(f"\nTime    : {elapsed:.2f} seconds")
    print(f"WPM     : {wpm}")
    print(f"Accuracy: {accuracy}%")
    return wpm

import random
sentence = random.choice(SENTENCES)
wpm = run_test(sentence)
best = load_best()

if wpm > best:
    print(f"New best score! {wpm} WPM")
    save_best(wpm)
else:
    print(f"Your best so far: {best} WPM")
```

Program 15 — CSV Student Report Generator

python

```
# Reads student marks from a CSV and generates a report file

import csv
import os

INPUT_FILE  = "marks.csv"
OUTPUT_FILE = "report.txt"
```

```
def create_sample_csv():
    # Create a sample CSV to work with
    rows = [
        ["Name",    "Math", "Science", "English"],
        ["Alice",   92,     88,        95],
        ["Bob",     70,     65,        72],
        ["Carol",   55,     60,        50],
        ["David",   85,     90,        88],
    ]
    with open(INPUT_FILE, "w", newline="") as f:
        csv.writer(f).writerows(rows)

def generate_report():
    with open(INPUT_FILE, "r") as f:
        reader = list(csv.DictReader(f))

    with open(OUTPUT_FILE, "w") as out:
        out.write("STUDENT REPORT\n" + "=" * 35 + "\n")
        for row in reader:
            marks = [int(row["Math"]), int(row["Science"]), int(row["English"])]
            avg   = sum(marks) / len(marks)
            grade = "A" if avg >= 80 else "B" if avg >= 65 else "C"
            line  = f"{row['Name']:<10} Avg: {avg:>5.1f}  Grade: {grade}\n"
            out.write(line)
            print(line, end="")

    print(f"\nReport saved to {OUTPUT_FILE}")

create_sample_csv()
generate_report()
```

Program 16 — Decorator-Based Function Logger

python

```
# A decorator that logs every function call with arguments and result

import os
from datetime import datetime

LOG_FILE = "function_log.txt"

def log_call(func):
    def wrapper(*args, **kwargs):
        result    = func(*args, **kwargs)
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        log_entry = (
            f"[{timestamp}] "
            f"{func.__name__}(args={args}, kwargs={kwargs}) "
            f"→ {result}\n"
        )
        # Print to screen
        print(log_entry, end="")
        # Save to file
        with open(LOG_FILE, "a") as f:
```

```
            f.write(log_entry)
        return result
    return wrapper

# Apply the decorator to any function
@log_call
def add(a, b):
    return a + b

@log_call
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

@log_call
def power(base, exp):
    return base ** exp

# Call the functions — everything gets logged automatically
add(5, 3)
greet("Alice")
greet("Bob", greeting="Hi")
power(2, 8)

print(f"\nAll calls saved to {LOG_FILE}")
```

Program 17 — Recursive Folder Size Calculator

python

```
# Walks through a folder and calculates total size using recursion

import os

def get_folder_size(folder_path):
    total = 0
    for item in os.listdir(folder_path):
        item_path = os.path.join(folder_path, item)
        if os.path.isfile(item_path):
            total += os.path.getsize(item_path)
        elif os.path.isdir(item_path):
            # Recursively calculate size of sub-folders
            total += get_folder_size(item_path)
    return total

def format_size(bytes_size):
    # Convert bytes to a readable unit
    for unit in ["B", "KB", "MB", "GB"]:
        if bytes_size < 1024:
            return f"{bytes_size:.2f} {unit}"
        bytes_size /= 1024

def show_folder_info(folder_path="."):
    size = get_folder_size(folder_path)
    files = sum(len(f) for _, _, f in os.walk(folder_path))
    print(f"Folder : {os.path.abspath(folder_path)}")
```

```
    print(f"Files  : {files}")
    print(f"Size   : {format_size(size)}")

show_folder_info(".")
```

Program 18 — Word Scramble Game

python

```
# Scrambles a word and lets user guess — tracks score, saves to file

import random
import json
import os

SCORE_FILE = "scramble_score.json"

WORDS = ["python", "function", "module", "variable",
         "recursion", "decorator", "iterator", "exception"]

def scramble(word):
    letters = list(word)
    while True:
        random.shuffle(letters)
        scrambled = "".join(letters)
        if scrambled != word:   # make sure it's actually scrambled
            return scrambled

def load_score():
    if os.path.exists(SCORE_FILE):
        with open(SCORE_FILE, "r") as f:
            return json.load(f)
    return {"wins": 0, "losses": 0}

def save_score(score):
    with open(SCORE_FILE, "w") as f:
        json.dump(score, f)

def play():
    score = load_score()
    word  = random.choice(WORDS)
    jumbled = scramble(word)

    print(f"\nUnscramble this word: {jumbled.upper()}")
    guess = input("Your answer: ").strip().lower()

    if guess == word:
        print("Correct!")
        score["wins"] += 1
    else:
        print(f"Wrong! The word was: {word}")
        score["losses"] += 1

    save_score(score)
    print(f"Score → Wins: {score['wins']}  Losses: {score['losses']}")
```

```
play()
```

Program 19 — Multi-Function Calculator with History

python

```
# Calculator with add/sub/mul/div — saves every calculation to history

import json
import os

HISTORY_FILE = "calc_history.json"

# Core math functions
def add(a, b):      return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return round(a / b, 4)

OPERATIONS = {
    "+": add,
    "-": subtract,
    "*": multiply,
    "/": divide,
}

def save_history(expression, result):
    history = []
    if os.path.exists(HISTORY_FILE):
        with open(HISTORY_FILE, "r") as f:
            history = json.load(f)
    history.append({"expr": expression, "result": result})
    with open(HISTORY_FILE, "w") as f:
        json.dump(history, f, indent=2)

def show_history():
    if not os.path.exists(HISTORY_FILE):
        print("No history yet.")
        return
    with open(HISTORY_FILE, "r") as f:
        history = json.load(f)
    print("\n--- Calculation History ---")
    for entry in history:
        print(f"  {entry['expr']} = {entry['result']}")

def calculate(a, op, b):
    if op not in OPERATIONS:
        print("Invalid operator.")
        return
    try:
        result = OPERATIONS[op](a, b)
```

```
        expr   = f"{a} {op} {b}"
        print(f"{expr} = {result}")
        save_history(expr, result)
    except ValueError as e:
        print(f"Error: {e}")

# Demo calculations
calculate(10, "+", 5)
calculate(20, "/", 4)
calculate(7,  "*", 8)
calculate(9,  "/", 0)
show_history()
```

Program 20 — Mini Student Management System

python

```
# Full system: add, view, search, delete students — all saved in JSON

import json
import os

FILE = "students.json"

def load():
    return json.load(open(FILE)) if os.path.exists(FILE) else []

def save(data):
    with open(FILE, "w") as f:
        json.dump(data, f, indent=2)

def add_student(name, age, grade, marks):
    students = load()
    # Check for duplicate names
    if any(s["name"].lower() == name.lower() for s in students):
        print(f"'{name}' already exists.")
        return
    students.append({"name": name, "age": age, "grade": grade, "marks": marks})
    save(students)
    print(f"Student '{name}' added.")

def view_all():
    students = load()
    if not students:
        print("No students found.")
        return
    print(f"\n{'Name':<12} {'Age':<5} {'Grade':<7} {'Marks'}")
    print("-" * 35)
    for s in students:
        print(f"{s['name']:<12} {s['age']:<5} {s['grade']:<7} {s['marks']}")

def search_student(name):
    students = load()
    found = [s for s in students if name.lower() in s["name"].lower()]
    if found:
```

```
        for s in found:
            print(f"\nFound: {s}")
    else:
        print("No student found.")

def delete_student(name):
    students = load()
    updated = [s for s in students if s["name"].lower() != name.lower()]
    if len(updated) == len(students):
        print("Student not found.")
    else:
        save(updated)
        print(f"'{name}' removed.")

def top_student():
    students = load()
    if students:
        best = max(students, key=lambda s: s["marks"])
        print(f"\nTop student: {best['name']} with {best['marks']} marks")

# Demo
add_student("Alice",  20, "A", 92)
add_student("Bob",    21, "B", 75)
add_student("Carol",  19, "A", 88)
add_student("David",  22, "C", 60)
view_all()
search_student("ali")
top_student()
delete_student("Bob")
view_all()
```